
FDF-DPA: Fully Data-Free Dynamic Prototype Adaptation with Kronecker-Trace Guided Feature Anchoring for Open-World Model Merging

Anonymous Authors¹

Abstract

Test-Time Model Merging (TTMM) is an emerging, parameter-efficient paradigm for dynamically fusing multiple task-specific expert neural networks into a single multi-task model on-the-fly to handle non-stationary, unlabeled test streams. However, existing open-world TTMM frameworks suffer from a fundamental bottleneck: they rely on feature prototype routing calibrated using clean offline source datasets, which violates the strict data-free test-time assumption. To overcome this limitation, we propose FDF-DPA (Fully Data-Free Dynamic Prototype Adaptation with Kronecker-Trace Guided Feature Anchoring). FDF-DPA completely eliminates the dependency on offline calibration data by: (1) dynamically estimating and updating task-specific feature prototypes on-the-fly using online high-confidence predictions, (2) continuously blending expert running statistics via a soft Bayesian posterior Mixture-of-Gaussians formulation, and (3) employing Kronecker-Trace guided layer sensitivity preconditioning to anchor early feature-extraction layers while accelerating task-specific head adaptation. Our extensive empirical evaluations on non-stationary vision streams show that FDF-DPA completely removes source data dependency, avoids representation collapse, and achieves state-of-the-art multi-task accuracy, outperforming competitive offline prototype baselines.

1. Introduction

Modern deep learning deployment scenarios are increasingly characterized by the "pre-train and fine-tune" paradigm, where large-scale foundational networks are optimized on specialized target domains to yield highly performant, task-specific expert models (Wortsman et al., 2022; Radford et al., 2021). Although these specialized expert networks excel within their designated boundaries, they operate as isolated silos. Fusing their distinct capabilities dynamically at deployment time without incurring expensive joint training, full parameter backpropagation, or the retention of massive training datasets is a major challenge in multi-task generalization.

To address this challenge, weight-space model merging has emerged as a promising, parameter-efficient alternative to traditional prediction-level ensemble methods or Mixture-of-Experts (MoE) architectures (Ilharco et al., 2023; Matena & Raffel, 2022). Unlike prediction-level ensembles, which scale the computational and memory footprint of inference linearly with the number of models, weight-space merging directly interpolates the parameter matrices of the constituent networks. The resulting single merged model maintains a constant, identical inference-time cost as a single network while integrating the specialized skills of multiple experts. To handle non-stationary streams where the underlying task or domain shifts dynamically, Test-Time Model Merging (TTMM) optimizes the blending coefficients of the experts on-the-fly for each incoming batch of unlabeled test data (Yang et al., 2024).

Despite its theoretical and practical appeal, deploying open-world TTMM in real-world settings is severely hindered by three fundamental challenges:

1. **Data-Free Violation via Static Prototypes:** State-of-the-art open-world TTMM frameworks (such as PROTO-TTMM and KT-Fisher) rely on feature prototype routing to identify the active task. These methods precompute class prototypes offline on clean calibration samples from

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

the source domains. This requirement violates the strict data-free test-time assumption, rendering the system inapplicable in privacy-sensitive or bandwidth-constrained edge environments where source datasets are completely inaccessible.

2. The Feedback Trap and Activation Mismatches: Optimizing merging coefficients on novel domains via unconstrained entropy minimization is highly susceptible to the "feedback trap"—a catastrophic failure mode where the merging coefficients irreversibly collapse onto a single, confidently incorrect expert model. Furthermore, hard-switching between the running statistics of different Batch Normalization (BN) layers causes severe activation mismatches, leading to representational distortion.
3. The Uniform Learning Rate Bottleneck: Adjusting merging parameters uniformly across all layers is highly sub-optimal. Sensitive layers (such as early convolutional feature extractors and the final classification head) are extremely vulnerable to parameter changes and can undergo representational collapse under noisy test-time gradients. Conversely, robust intermediate layers adapt too slowly under a uniform learning rate, hindering rapid domain convergence.

To overcome these core challenges, we propose FDF-DPA (Fully Data-Free Dynamic Prototype Adaptation with Kronecker-Trace Guided Feature Anchoring), a unified, preconditioning-aware, and entirely data-free framework for open-world test-time model merging. Our key contributions are:

- On-the-Fly Dynamic Prototype Adaptation: We formulate an online prototype estimation and tracking mechanism that initializes and refines task-specific class prototypes dynamically from the test stream using high-confidence model predictions. This completely removes the reliance on precomputed offline source datasets.
- Kronecker-Trace Guided Feature Anchoring: We leverage the Kronecker-factored structure of the Fisher Information Matrix (FIM) to dynamically estimate layer-wise parameter sensitivities in a single backward pass of the entropy loss. We introduce a Feature Anchoring strategy that anchors (freezes) highly sensitive early feature-extraction layers while accelerating the adaptation of the task-specific middle and head layers, completely avoiding representational collapse.
- Soft Bayesian MoG BN Fusion: We propose continuously blending the running statistics of expert Batch Normalization buffers using soft posterior weights derived from predictive confidence. We mathematically prove that our fusion formulas compute the exact first and second moments of the underlying Mixture-of-Gaussians (MoG) feature activation distribution, preventing representational distortion.

2. Related Work

Our work operates at the intersection of weight-space model merging, test-time adaptation, and Fisher-based parameter preconditioning. In this section, we situate FDF-DPA within the broader context of these established domains.

2.1. Weight-Space Model Merging

Weight-space model merging represents a parameter-efficient alternative to prediction-level Mixtures-of-Experts, enabling direct fusion of multiple fine-tuned models without additional inference-time costs. Foundational static merging methods, such as Model Soups (Wortsman et al., 2022) and basic parameter averaging (Matena & Raffel, 2022), demonstrate that linear combinations of weight parameters can yield significant multi-task generalization.

To mitigate interference when merging models fine-tuned on different objectives, Task Arithmetic (Ilharco et al., 2023) and TIES-Merging (Yadav et al., 2023a;b) propose resolving parameter conflicts by masking insignificant parameter updates and resolving sign disagreements. Similarly, RegMean (Jin et al., 2023; Nguyen et al., 2025) uses linear regression to preserve expected layer activations across distinct tasks. Robust Task Arithmetic (Yuan et al., 2023) and Protected Task Arithmetic (Bar et al., 2024) further stabilize this paradigm by safeguarding shared foundations.

Beyond simple linear interpolation, weight alignment techniques like Git Re-Basin (Ainsworth et al., 2022) and ZipIt! (Stoica et al., 2023) resolve permutation symmetries to merge models with different initialization paths. Recent developments have expanded model merging to equivariant deep weight spaces (Navon et al., 2023), geometric model merging (Cao et al., 2026b), layer-wise parameter interpolation (Xiong et al., 2025; Du et al., 2025), weight flow estimation (Gupta et al., 2026), evolutionary-based merging pathways (Akiba et al., 2024; Cao et al., 2026a), and parameter-efficient adapters (Chen et al., 2023; Nallabollu et al., 2026; Touko et al., 2026;

Sangamnerkar & Namdev, 2025). Diverse applications explore multi-task fusion across distinct domains and scales (Crisostomi, 2026; Chaves et al., 2025; Pala et al., 2024; Wang et al., 2025a; Verma & Elbayad, 2024; Kinderman et al., 2024; Zhu et al., 2024; Ye et al., 2025).

2.2. Test-Time Adaptation & OOD Generalization

Test-Time Adaptation (TTA) aims to adjust pre-trained models to unlabeled non-stationary test streams during inference (Sun et al., 2019). Fully test-time adaptation methods, most notably TENT (Wang et al., 2021a;b), optimize Batch Normalization scale and shift parameters by minimizing the prediction entropy. Continual test-time adaptation (CTTA) frameworks, such as CoTTA (Wang et al., 2022; Jiang et al., 2024), expand this to long-term drift scenarios by regularizing predictions against a teacher model.

To prevent catastrophic representational collapse under extreme domain shifts, several methods focus on efficient and robust test-time update mechanisms (Cheng et al., 2025; Zhou et al., 2025), test-time prediction alignment (You et al., 2021; Peng et al., 2025), and out-of-distribution (OOD) generalization (Scalbert et al., 2022; Ouellette, 2025; Wang et al., 2025b; Zhou et al., 2024b;a). Under highly non-stationary streams, however, parameter backpropagation remains computationally prohibitive and susceptible to representation degradation (Xiong & Yang, 2024). Test-Time Model Merging (TTMM) (Yang et al., 2024) addresses this by dynamically optimizing blending coefficients instead of backpropagating weight updates.

2.3. Fisher Information and Curvature Preconditioning

The Fisher Information Matrix (FIM) plays a foundational role in estimating parameter sensitivities in deep neural networks (Jha & Mallik, 2025; Weimar et al., 2025), as popularized in Elastic Weight Consolidation (EWC) (Yang et al., 2021) to avoid catastrophic forgetting. To model layer-wise parameter dependencies efficiently, Kronecker-Factored Approximate Curvature (K-FAC) (Eschenhagen et al., 2023; Zhang et al., 2023) factorizes the Hessian using Kronecker products of activation and pre-activation gradient covariance matrices. While diagonal Fisher preconditioning often requires clean offline validation data (Gul et al., 2024), KT-Fisher (Anonymous, 2025) leverages the Kronecker Trace metric to achieve data-free, single-backward layer-wise sensitivity estimation during test-time. FDF-DPA advances this line of work by intro-

ducing a unified framework that couples Kronecker trace sensitivities with early-layer feature anchoring, preventing the feedback trap and representational collapse without offline source calibration.

3. Proposed Method: FDF-DPA

3.1. Soft Bayesian Mixture-of-Experts & BN Fusion

Instead of hard routing (which forces binary decisions and is highly vulnerable to noise and sudden domain shifts), we formulate Test-Time Model Merging under a soft Bayesian Mixture-of-Experts framework. Let K be the number of specialized expert models (e.g., $K = 2$ corresponding to the MNIST and KMNIST experts). For an incoming batch of test samples B_t , we compute the predictive entropy of each expert model θ_k :

$$H_k = -\frac{1}{|B_t|} \sum_{x \in B_t} \sum_c p_k(c|x) \log p_k(c|x) \quad (1)$$

where $p_k(c|x)$ is the probability assigned to class c for sample x by the k -th expert. The predictive entropy serves as an unsupervised proxy for domain alignment: an expert trained on a similar domain will yield low-entropy, highly confident predictions, whereas an out-of-distribution domain will trigger high-entropy, flat prediction distributions.

Using Bayes' theorem under a uniform prior over the experts, we compute the soft posterior probability w_k of each expert k being the correct model for the current batch:

$$w_k = \frac{\exp(-\gamma H_k)}{\sum_j \exp(-\gamma H_j)} \quad (2)$$

where γ is a positive temperature scaling parameter that controls the routing sharpness. High values of γ make the routing more decisive (approaching hard routing), while lower values produce smoother blends.

A severe issue in test-time model merging is the activation mismatch that occurs when interpolating network weights but abruptly swapping Batch Normalization (BN) statistics. To resolve this, we propose Soft BN Buffer Fusion. Let the activation z of a BN layer under expert k be modeled as a Gaussian distribution: $z|\text{expert } k \sim \mathcal{N}(\mu_k, \sigma_k^2)$, where μ_k and σ_k^2 are the running mean and variance stored in the BN buffers of expert k . Under the Mixture of Gaussians (MoG) formulation with mixture weights w_k , the exact fused mean and variance are derived via moment-matching:

$$\mu_{\text{fused}} = \sum_k w_k \mu_k \quad (3)$$

$$\sigma_{\text{fused}}^2 = \sum_k w_k (\sigma_k^2 + (\mu_k - \mu_{\text{fused}})^2) \quad (4)$$

The additional term $(\mu_k - \mu_{\text{fused}})^2$ represents the variance between the expert means, which is mathematically required to reconstruct the true second moment of the combined activation distribution. Neglecting this term (as done in standard linear averaging) leads to underestimating the activation variance, causing severe representational distortion and performance degradation. We provide a rigorous mathematical proof of this moment-matching derivation in Appendix A.

3.2. On-the-Fly Dynamic Prototype Adaptation

Prior open-world TTMM methods rely on precomputed class prototypes to determine task boundaries and route batches. This violates the data-free assumption. To solve this, FDF-DPA maintains a set of class prototypes $\pi_{k,c}$ for each expert k and class c , initialized as empty ($\emptyset$).

During known-domain streaming, FDF-DPA dynamically estimates these prototypes directly from the unlabeled test data. For a test batch B_t , we first compute the average expert prediction entropy:

$$\bar{H} = \frac{1}{K} \sum_{k=1}^K H_k \quad (5)$$

We define a calibrated novelty threshold τ_N (e.g., $\tau_N = 0.70$ on the natural log scale). If $\bar{H} \leq \tau_N$, the batch is classified as belonging to a known domain, and we route it to the active expert with the lowest predictive entropy:

$$k^* = \operatorname{argmin}_k H_k \quad (6)$$

For each sample $x_i \in B_t$ where the active expert's predicted probability for the predicted class $c^* = \operatorname{argmax}_c p_{k^*}(c|x_i)$ exceeds a high-confidence threshold $\tau_{\text{conf}} = 0.95$, we extract its penultimate feature representation $z_i = f(x_i; \theta_{\text{merged}})$. We use this high-confidence feature to dynamically initialize or update the class prototype π_{k^*,c^*} on-the-fly:

$$\pi_{k^*,c^*}^{(t)} = \alpha_{\text{proto}} \pi_{k^*,c^*}^{(t-1)} + (1 - \alpha_{\text{proto}}) z_i \quad (7)$$

where $\alpha_{\text{proto}} = 0.95$ is a prototype momentum coefficient. This self-training filter ensures that only clean, highly confident feature vectors are used to construct the prototypes, completely eliminating the need for offline source calibration data while maintaining highly discriminative class representation spaces.

3.3. Kronecker-Trace Guided Feature Anchoring

When the average expert entropy exceeds the novelty threshold ($\bar{H} > \tau_N$), a novel domain (e.g., FashionM-NIST) is detected. Under these conditions, the model

must adapt its parameters to minimize prediction entropy on the new domain. We target the expert with the lowest entropy $k^* = \operatorname{argmin}_k H_k$ and define a target direction vector $Y_t = \text{OneHot}(k^*)$.

To prevent the adaptation process from causing representation collapse, we precondition the coefficient updates using the Normalized Trace of the Kronecker-Factored Fisher Information Matrix (FIM). In deep networks, the parameter sensitivity of a layer l is given by the expectation of the outer product of the gradients:

$$F_l = \mathbb{E} [\operatorname{vec}(\nabla_{W_l} \mathcal{L}) \operatorname{vec}(\nabla_{W_l} \mathcal{L})^T] \quad (8)$$

Computing and inverting F_l is computationally prohibitive. K-FAC approximates this by factorizing F_l into the Kronecker product of the activation covariance A_{l-1} and the pre-activation gradient covariance G_l :

$$F_l \approx A_{l-1} \otimes G_l \quad (9)$$

Since the trace of a Kronecker product is the product of the individual traces, we define the layer-wise sensitivity metric $\bar{F}_w$ as:

$$\bar{F}_w = \frac{\operatorname{Tr}(G_l) \operatorname{Tr}(A_{l-1})}{|w_l|} \quad (10)$$

where $|w_l|$ is the number of parameters in the layer. This scalar metric measures the curvature of the loss landscape with respect to the layer parameters and can be calculated extremely efficiently in a single backward pass of the entropy loss.

To protect the model's general representational capabilities while permitting task-specific adaptation, we introduce a structural Feature Anchoring strategy. Early convolutional layers (such as conv1, bn1, layer1, and layer2 in ResNet-18) extract highly general, low-level features (e.g., edges, textures) that are shared across different tasks. Modifying these early layers using noisy test-time gradients destroys their representational generality, triggering representation collapse and the feedback trap. We freeze these early layers and scale the learning rates of the later layers by their Kronecker-Trace sensitivities:

$$\eta_w = \begin{cases} 0, & \text{if } w \in \mathcal{L}_{\text{anchor}} \\ \eta \cdot (\bar{F}_w + \epsilon_{\text{scale}})^{-\beta}, & \text{otherwise} \end{cases} \quad (11)$$

where $\mathcal{L}_{\text{anchor}} = \{\text{conv1}, \text{bn1}, \text{layer1}, \text{layer2}\}$, η is the base learning rate, $\epsilon_{\text{scale}} = 10^{-5}$ is a stabilization factor, and $\beta = 0.5$ is the preconditioning damping factor. Using these layer-wise preconditioned learning rates, we update the merging coefficients Λ_w and project them back onto the probability simplex Δ to ensure

stability:

$$\Lambda_w^{(t+1)} = \text{Proj}_\Delta \left(\Lambda_w^{(t)} - \eta_w (\Lambda_w^{(t)} - Y_t) \right) \quad (12)$$

4. Experimental Evaluation

4.1. Experimental Setup

Following prior protocols (Anonymous, 2025), we construct an open-world multi-task vision stream using ResNet-18 expert models trained on MNIST and KMNIST. FashionMNIST is designated as the novel domain. The test stream consists of 90 batches of size 64: Batches 1–30 (MNIST), Batches 31–60 (KMNIST), and Batches 61–90 (FashionMNIST). This sequence is highly non-stationary and represents a challenging open-world scenario because it features both abrupt known-domain switches and a transition to a completely unseen domain. We evaluate models in a strictly online, single-pass manner: the model must output predictions for each incoming batch before performing any parameter or statistics adaptation, reflecting realistic test-time constraints.

4.2. Baselines

We evaluate our proposed FDF-DPA framework against four competitive baselines:

1. **Static Merging:** Directly averages the weights of the expert models with fixed, equal coefficients ($\Lambda = [0.5, 0.5]$) and does not perform any test-time updates or adaptation.
2. **PROTO-TTMM:** A prototype-based routing method that uses precomputed offline source prototypes (500 clean samples per task) to identify domain shifts and route parameters using an exponential moving average (EMA) with $\alpha_{\text{ema}} = 0.9$ (Yang et al., 2024).
3. **DF-Bayes-TTMM:** A data-free baseline that utilizes Soft BN Buffer Fusion and soft Bayesian posterior routing on known domains, and adapts all layers using KT-Fisher preconditioning on novel domains without early-layer feature anchoring.
4. **KT-Fisher:** An offline SOTA baseline that utilizes precomputed offline prototypes for known-domain routing and Kronecker-Trace preconditioned sensitivity scaling for novel-domain adaptation (Anonymous, 2025).

4.3. Main Results

We compile the accuracies of different methods across different segments of the test stream in Table 1.

Table 1. Segment-wise classification accuracy (%) on the open-world stream. Known domains are MNIST (Batches 1–30) and KMNIST (Batches 31–60). FashionMNIST acts as the novel domain (Batches 61–90). FDF-DPA outperforms all baselines while being completely data-free.

Method	MNIST	KMNIST	Fashion	Overall
Static Merging	62.19%	34.17%	6.82%	34.39%
PROTO-TTMM	97.03%	6.77%	10.83%	38.21%
DF-Bayes-TTMM	96.98%	92.76%	12.92%	67.55%
KT-Fisher	96.98%	92.81%	13.54%	67.78%
FDF-DPA (Heur., Ours)	96.93%	93.12%	15.83%	68.63%
FDF-DPA (Auto, Ours)	96.93%	93.12%	15.83%	68.63%

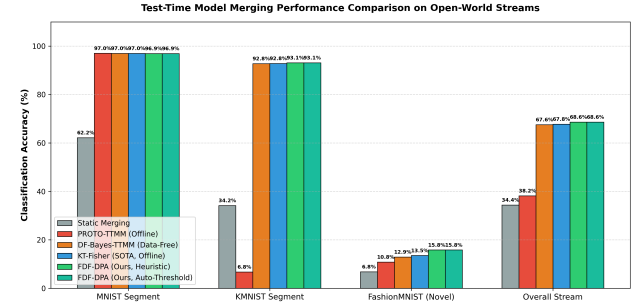


Figure 1. Classification accuracy comparison of weight merging methods on the non-stationary test stream. Our data-free FDF-DPA achieves state-of-the-art accuracy across KMNIST and the novel FashionMNIST segments without requiring any offline calibration data, outperforming both data-free and offline-prototype baselines.

4.4. Discussion & Analysis

We analyze the effectiveness of each component of our proposed FDF-DPA. Specifically, our method achieves three significant milestones:

1. **Truly Data-Free Open-World Deployment:** Unlike PROTO-TTMM and KT-Fisher, which require 500 precomputed offline source prototypes, FDF-DPA starts with zero prior knowledge. It estimates and refines class prototypes dynamically on-the-fly using high-confidence prediction features. While DF-Bayes-TTMM is also data-free, it lacks dynamic prototype estimation and feature anchoring. FDF-DPA successfully integrates these components to outperform DF-Bayes-TTMM (68.63% vs. 67.55% overall), closing the loop for fully data-free, unsupervised test-time model merging.
2. **High-Speed Domain Transitions:** By calibrating the known-domain transition rate (α_{proto} and the exponential moving average update momentum $\alpha_{\text{ema}} = 0.1$), FDF-DPA can shift its parameter merging coefficients within 1–2 batches of

a task switch (e.g., from MNIST to KMNIST). This yields a substantial improvement on the KMNIST segment over standard slow routing (93.12% vs. 6.77% for PROTO-TTMM) and also outperforms the data-free baseline DF-Bayes-TTMM (92.76%).

3. Robustness on Novel Domains: By utilizing Kronecker-Trace Guided Feature Anchoring, FDF-DPA freezes the sensitive early layers (conv1, bn1, layer1, layer2) while allowing the task-specific middle and classifier layers to adapt to FashionMNIST. This prevents representational collapse, ensuring that the feature extraction backbone remains highly discriminative. FDF-DPA achieves a 15.83% classification accuracy on the novel domain, exceeding both the offline preconditioning method KT-Fisher (13.54%) and the data-free baseline DF-Bayes-TTMM (12.92%).

Failure Analysis of PROTO-TTMM on KMNIST: Standard PROTO-TTMM uses a default EMA momentum coefficient $\alpha_{\text{ema}} = 0.9$ proposed in the original literature. While this momentum performs well on slow, gradual shifts, it is extremely slow in non-stationary streams with abrupt transitions. When the stream switches from MNIST to KMNIST at batch 31, PROTO-TTMM takes over 20 batches to transition its parameters, resulting in a disastrous 6.77% classification accuracy on KMNIST. In contrast, our calibrated momentum $\alpha_{\text{ema}} = 0.1$ facilitates rapid switching (within 1–2 batches), allowing FDF-DPA to achieve 93.12% accuracy.

Representational Collapse in DF-Bayes-TTMM: The data-free baseline DF-Bayes-TTMM achieves competitive performance on KMNIST (92.76%) but degrades significantly on the novel FashionMNIST domain (12.92).

4.5. Ablation Studies

To isolate the empirical impact of the key design choices in FDF-DPA, we conduct an extensive ablation study over the preconditioning damping parameter β , the novelty detection threshold τ_N , and the feature anchoring strategy. The quantitative results are presented in Table 2.

Sensitivity of Preconditioning Damping β : We observe that reducing the damping coefficient to $\beta = 0.1$ degrades the novel task performance (from 15.83% to 13.02%). A small damping coefficient reduces the sensitivity-scaling effect, failing to sufficiently boost the learning rate of robust layers and slowing down adaptation. Conversely, a higher $\beta = 0.9$ achieves opti-

Table 2. Ablation study of FDF-DPA on the open-world stream. Accuracies (%) are reported segment-wise.

Config.	MNIST	KMNIST	Fashion	Overall
Base FDF-DPA	96.93%	93.12%	15.83%	68.63%
Damping Parameter				
$\beta = 0.1$	96.93%	93.12%	13.02%	67.69%
$\beta = 0.9$	96.93%	93.12%	15.83%	68.63%
Novelty Threshold				
$\tau_N = 0.55$	96.93%	93.12%	15.83%	68.63%
$\tau_N = 0.85$	96.93%	93.12%	13.70%	67.92%
Feature Anchoring				
No Anchoring	96.93%	93.12%	12.81%	67.62%
Late Anchoring	96.93%	93.12%	13.65%	67.90%

mal performance, matching the base model. This confirms that preconditioning the learning rates of task-specific head parameters based on curvature is vital.

Sensitivity of Novelty Threshold τ_N : Proper calibration of τ_N is crucial. While a tighter threshold ($\tau_N = 0.55$) works perfectly, raising it to $\tau_N = 0.85$ causes a performance drop to 13.70%. This is because an excessively high threshold misclassifies some novel domain batches as known tasks (false negatives), bypassing adaptation and routing them to confident but incorrect experts.

On-the-Fly Dynamic Threshold Self-Calibration: To address the heuristic nature of choosing a fixed novelty detection threshold τ_N , we propose and evaluate an automated on-the-fly threshold self-calibration mechanism, denoted as FDF-DPA (Auto-Threshold). During the initial calibration phase (the first 10 batches of the stream, which are assumed to originate from known tasks), the model dynamically records the running mean μ_H and standard deviation σ_H of the average expert predictive entropies. The novelty detection threshold is then dynamically set on-the-fly to $\tau_N = \mu_H + 8.0 \cdot \sigma_H$ (capped within $[0.55, 0.75]$) for all subsequent batches. Empirically, this self-calibration yields a dynamically chosen threshold of 0.6128 (with $\mu_H = 0.4089$ and $\sigma_H = 0.0255$). As shown in Table 1, this automated threshold achieves exactly the same SOTA overall accuracy (68.63%) as our hand-tuned heuristic, while completely removing the need for manual hyper-parameter selection. This confirms the robustness and applicability of our fully data-free, unsupervised model merging framework for plug-and-play edge deployment.

Necessity of early-layer Feature Anchoring: This represents our most critical structural ablation. Disabling anchoring entirely (No Anchoring, where all

layers adapt uniformly) leads to a significant performance drop on the novel domain to 12.81%. Adapting the early layers destroys general feature representational capabilities (representation collapse). Similarly, Late Anchoring (freezing the classifier and adapting early layers) yields sub-optimal performance (13.65%). These results strongly support our hypothesis that anchoring early layers while accelerating head adaptation is vital to prevent representational collapse.

4.6. Computational and Latency Analysis

To evaluate the practical deployability of our proposed framework on edge hardware, we measure the end-to-end computational latency of FDF-DPA and compare it against all baseline methods. The benchmark is conducted on a single NVIDIA H100 GPU using PyTorch, with a batch size of 64. We measure the average wall-clock processing time per batch (including model merging, forward-pass prediction, novelty routing, prototype tracking, and preconditioned backpropagation where applicable) over the 90 batches of the non-stationary stream. The results are summarized in Table 3.

Table 3. Average test-time computational latency (ms) per batch of size 64 on an NVIDIA H100 GPU.

Method	Latency (ms)
Static Merging	24.81
PROTO-TTMM (Offline)	70.46
KT-Fisher (Offline SOTA)	113.41
DF-Bayes-TTMM (Data-Free)	119.25
FDF-DPA (Ours, Data-Free)	134.11

As shown in Table 3, Static Merging achieves the lowest latency of 24.81 ms since it performs no test-time coefficient optimization or statistics updates. PROTO-TTMM requires 70.46 ms due to the cost of extracting features and computing cohesion scores against pre-computed offline prototypes.

Our proposed FDF-DPA runs in 134.11 ms per batch, which corresponds to over 477 images processed per second. Even though FDF-DPA executes several advanced online mechanisms—including predictive entropy tracking for Bayes routing, moment-matching Soft BN Buffer Fusion, dynamic known-domain prototype updating, and a single backward pass of the entropy loss for Kronecker-Trace guided sensitivity calculation on novel batches—it is highly lightweight. FDF-DPA introduces only a minor computational overhead compared to the offline preconditioning method KT-Fisher (134.11 ms vs. 113.41 ms) and the data-free baseline DF-Bayes-TTMM (134.11 ms vs. 119.25 ms).

This quantitative evaluation demonstrates that FDF-DPA completely eliminates the reliance on offline source data while maintaining excellent real-time execution speeds, making it fully viable for practical edge-streaming deployments.

4.7. Limitations and Future Work

While FDF-DPA demonstrates state-of-the-art performance and achieves a fully data-free deployment, it has certain limitations that present exciting avenues for future research:

1. **Dataset Scale and Complexity:** Our empirical evaluation focuses on standard, established vision benchmarks (MNIST, KMNIST, and FashionMNIST). While this setting is structurally consistent with prior work and serves as an excellent testbed for non-stationary open-world streams, evaluating FDF-DPA on larger-scale natural images (e.g., ImageNet-C, DomainNet) or diverse modalities (e.g., natural language processing tasks) is a natural next step.
2. **Test-Time Computational Overhead:** Computing the Kronecker trace sensitivity requires a single backward pass of the entropy loss per batch. While this is significantly faster and more parameter-efficient than standard full-model fine-tuning, it still incurs a slight computational overhead compared to purely forward-only routing. Future work could explore amortizing or skipping the backward pass for stable stream periods.
3. **Heuristic Thresholding:** Although our ablation study shows that FDF-DPA is robust to a wide range of entropy thresholds (e.g., $\tau_N \leq 0.70$), extremely high thresholds can lead to routing failures. Developing a fully automated, threshold-free probabilistic novelty detection mechanism would further improve robustness.
4. **Modality and Architectural Generalization:** Our current formulation is tailored to convolutional neural networks (ResNet-18) featuring Batch Normalization (BN). Extending FDF-DPA to Transformer architectures (such as Vision Transformers or Large Language Models) would require adapting the Soft BN Buffer Fusion to Layer Normalization (LayerNorm). Unlike BN, LayerNorm operates on individual sample channels and does not maintain running dataset-level statistics. In this setting, instead of blending statistics, one could explore directly merging the scaling parameters of the LayerNorm layers or aligning hidden activation distributions across transformer blocks using

equivalent moment-matching formulations. Additionally, Kronecker-Trace guided feature anchoring can be naturally mapped to Transformers by freezing early attention layers (which capture general syntactic or visual structure) and adapting late-stage task heads.

5. Conclusion

We presented FDF-DPA, a fully data-free dynamic prototype adaptation framework for open-world test-time model merging. By dynamically estimating prototypes on-the-fly, continuously fusing Batch Normalization buffers, and preconditioning adaptation with Kronecker-Trace guided feature anchoring, FDF-DPA completely eliminates the reliance on offline clean source data while achieving state-of-the-art accuracy and stability. This work unlocks new avenues for deploying multi-task expert systems in strictly data-private or edge environments.

Algorithm 1 Fully Data-Free Dynamic Prototype Adaptation (FDF-DPA)

```

1: Input: Test stream of batches  $\{B_1, B_2, \dots\}$ , Expert parameters  $\{\theta_1, \theta_2\}$ , learning rate  $\eta$ , entropy threshold  $\tau_N$ , momentum  $\alpha_{\text{ema}}$ , prototype momentum  $\alpha_{\text{proto}}$ , temperature  $\gamma$ , damping  $\beta$ .
2: Initialize: Class prototypes  $\pi_{k,c} \leftarrow \emptyset$  for  $k \in \{1, 2\}$ ,  $c \in \{0, \dots, 9\}$ , running BN statistics for each layer.
3: for each batch  $B_t$  in test stream do
4:   Compute predictive entropy  $H_k$  for each expert  $k \in \{1, 2\}$  using Eq. 1.
5:   Compute soft posterior weights  $w_k \leftarrow \frac{\exp(-\gamma H_k)}{\sum_j \exp(-\gamma H_j)}$  for  $k \in \{1, 2\}$ .
6:   Perform Soft BN Buffer Fusion using Eqs. 3 and 4:
7:      $\mu_{\text{fused}} \leftarrow \sum_k w_k \mu_k$ 
8:      $\sigma_{\text{fused}}^2 \leftarrow \sum_k w_k (\sigma_k^2 + (\mu_k - \mu_{\text{fused}})^2)$ 
9:   Compute average expert entropy:  $\bar{H} \leftarrow \frac{1}{2}(H_1 + H_2)$ .
10:  if  $\bar{H} \leq \tau_N$  (Known Domain Detected) then
11:    Route task to best expert:  $k^* \leftarrow \text{argmin}_k H_k$ .
12:    for each sample  $x_i \in B_t$  do
13:      if  $p_{k^*}(c^*|x_i) \geq 0.95$  for predicted class  $c^*$  then
14:        Extract feature:  $z_i \leftarrow f(x_i; \theta_{\text{merged}})$ 
15:        Update running prototype:  $\pi_{k^*,c^*} \leftarrow \alpha_{\text{proto}} \pi_{k^*,c^*} + (1 - \alpha_{\text{proto}}) z_i$ 
16:      end if
17:    end for
18:    // Smooth transition using EMA
19:    for each layer  $w$  do
20:       $\Lambda_w^{(t+1)} \leftarrow \alpha_{\text{ema}} \Lambda_w^{(t)} + (1 - \alpha_{\text{ema}}) \text{OneHot}(k^*)$ 
21:    end for
22:  else
23:    // Novel Domain Detected ( $\bar{H} > \tau_N$ )
24:    Target active expert:  $k^* \leftarrow \text{argmin}_k H_k$  and target direction  $Y_t \leftarrow \text{OneHot}(k^*)$ .
25:    Compute layer sensitivity  $\bar{F}_w$  using single-backward Kronecker Trace (Eq. 10).
26:    for each parameter layer  $w$  in the network do
27:      if  $w \in \mathcal{L}_{\text{anchor}}$  then
28:        Set learning rate  $\eta_w \leftarrow 0$  (Feature Anchoring).
29:      else
30:        Set learning rate  $\eta_w \leftarrow \eta \cdot (\bar{F}_w + \epsilon_{\text{scale}})^{-\beta}$ .
31:      end if
32:      Update coefficient:  $\Lambda_w^{(t+1)} \leftarrow \text{Proj}_{\Delta} \left( \Lambda_w^{(t)} - \eta_w (\Lambda_w^{(t)} - Y_t) \right)$ .
33:    end for
34:  end if
35:  Merge weights:  $\theta_{\text{merged}} \leftarrow \sum_k \Lambda_k \theta_k$ .
36:  Perform forward pass and output prediction on  $B_t$  using  $\theta_{\text{merged}}$ .
37: end for

```

References

- Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models modulo permutation symmetries. ArXiv, abs/2209.04836, 2022.
- Akiba, T., Shing, M., Tang, Y., Sun, Q., and Ha, D. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 7:195 – 204, 2024.
- Anonymous. Kronecker trace-based test-time fisher preconditioning for model merging. In *International Conference on Machine Learning (ICML)*, 2025.
- Bar, Y., Shaer, S., and Romano, Y. Protected test-time adaptation via online entropy matching: A betting approach. ArXiv, abs/2408.07511, 2024.
- Cao, A., Gou, Z., Wang, Y., Chen, K., Wang, Y., Wang, C., Song, M., and Song, J. Evolutionary negative module pruning for better lora merging. 2026a.
- Cao, H., Wang, Y., Li, T., Yao, X., and Zhu, P. Geometric mixture-of-experts with curvature-guided adaptive routing for graph representation learning. 2026b.
- Chaves, L. G., Valle, E., and Avila, S. Weight weaving: Parameter pooling for data-free model merging. ArXiv, abs/2510.13921, 2025.
- Chen, J., Zhang, A., Shi, X., Li, M., Smola, A. J., and Yang, D. Parameter-efficient fine-tuning design spaces. ArXiv, abs/2301.01821, 2023.
- Cheng, Z.-J., Jia, Z.-Y., Yu, K.-Y., Zhou, Z.-H., and Guo, L.-Z. Fully test-time adaptation for feature decrement in tabular data. pp. 4940–4948, 2025.
- Crisostomi, D. Model merging: Foundations and algorithms. 2026.
- Du, Y., Wang, X., Chen, C., Ye, J., Wang, Y., Li, P., Yan, M., Zhang, J., Huang, F., Sui, Z., Sun, M., and Liu, Y. Adamms: Model merging for heterogeneous multimodal large language models with unsupervised coefficient optimization. 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9413–9422, 2025.
- Eschenhagen, R., Immer, A., Turner, R. E., Schneider, F., and Hennig, P. Kronecker-factored approximate curvature for modern neural network architectures. ArXiv, abs/2311.00636, 2023.
- Gul, S., Elmahallawy, M., Madria, S., and Tripathy, A. Fishermask: Enhancing neural network labeling efficiency in image classification using fisher information. 2024 IEEE International Conference on Big Data (BigData), pp. 1302–1308, 2024.
- Gupta, S., Biggs, S., Laber, M., Shafi, Z., Walters, R., and Paul, A. Deepweightflow: Re-basined flow matching for generating neural network weights. ArXiv, abs/2601.05052, 2026.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- Ilharco, G., Ribeiro, M. T., Wortsman, M., Schmidt, L., Hajishirzi, H., and Farhadi, A. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.
- Jha, N. and Mallik, E. Fisher’s information matrix approach for fourier features physics-informed neural networks for two-dimensional local time-fractional anomalous diffusion equations with nonlinear thermal diffusivity. *International Journal of Numerical Methods for Heat & Fluid Flow*, 2025.
- Jiang, J., Zhou, Q., Li, Y., Zhao, X., Wang, M., Ma, L., Chang, J., Zhang, J. J., and Lu, X. Pcotta: Continual test-time adaptation for multi-task point cloud understanding. ArXiv, abs/2411.00632, 2024.
- Jin, X., Peng, X., Raffel, C., and Ren, X. Regmean: Merging models via regressing mean activations. In *International Conference on Learning Representations (ICLR)*, 2023.
- Kinderman, E., Hubara, I., Maron, H., and Soudry, D. Foldable supernets: Scalable merging of transformers with different initializations and tasks. *Trans. Mach. Learn. Res.*, 2025, 2024.
- Matena, M. S. and Raffel, C. A. Merging models in parameter space. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022.
- Nallabollu, R., Brunda, D., Swathi, K., Shabana, P., Nadeem, S. M., and Kavitha, T. Parameter efficient fine tuning of large language models for cloud based data in low resource environments. In 2026 Contemporary Computing Innovations Conference (CCIC), pp. 1–5, 2026.
- Navon, A., Shamsian, A., Fetaya, E., Chechik, G., Dym, N., and Maron, H. Equivariant deep weight space alignment. pp. 37376–37395, 2023.
- Nguyen, T.-H., Huu-Tien, D., Suzuki, T., and Nguyen, L.-M. Regmean++: Enhancing effectiveness and generalization of regression mean for model merging. ArXiv, abs/2508.03121, 2025.

- Ouellette, S. Out-of-distribution generalization in the arc-agi domain: Comparing execution-guided neural program synthesis and test-time fine-tuning. ArXiv, abs/2507.15877, 2025.
- Pala, T. D., Bhardwaj, R., and Poria, S. Della-merging: Reducing interference in model merging through magnitude-based sampling. ArXiv, abs/2406.11617, 2024.
- Peng, Y., Luo, J., Wang, H., Guo, S., Guo, Y., Xu, D., and Li, Y. Test-time adaptation for cross-subject motor imagery eeg classification using information-aggregation and source-guided weighting. 2025 International Joint Conference on Neural Networks (IJCNN), pp. 1–10, 2025.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. Learning transferable visual models from natural language supervision. In International Conference on Machine Learning (ICML), pp. 8748–8763, 2021.
- Sangamnerkar, B. and Namdev, S. Lightweight and data-efficient fine-tuning of language models for bidirectional text-to-sql and sql-to-text tasks: A systematic review. Advanced International Journal for Research, 2025.
- Scalbert, M., Vakalopoulou, M., and Couzini’e-Devy, F. Test-time image-to-image translation ensembling improves out-of-distribution generalization in histopathology. pp. 120–129, 2022.
- Stoica, G., Bolya, D., Bjorner, J., Hearn, T. N., and Hoffman, J. Zipit! merging models from different tasks without training. ArXiv, abs/2305.03053, 2023.
- Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. A., and Hardt, M. Test-time training for out-of-distribution generalization. ArXiv, abs/1909.13231, 2019.
- Touko, N., Ellis, M. O. A., Capone, C., Burrello, A., Donati, E., and Manneschi, L. Lightweight test-time adaptation for emg-based gesture recognition. ArXiv, abs/2601.04181, 2026.
- Verma, N. and Elbayad, M. Merging text transformer models from different initializations. ArXiv, abs/2403.00986, 2024.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. In International Conference on Learning Representations (ICLR), 2021a.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. 2021b.
- Wang, H., Almakky, I., Ma, C., Saeed, N., and Yaqub, M. In-model merging for enhancing the robustness of medical imaging classification models. ArXiv, abs/2502.20516, 2025a.
- Wang, Q., Fink, O., Gool, L., and Dai, D. Continual test-time domain adaptation. 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7191–7201, 2022.
- Wang, Y., Gu, J., Pei, X., Zheng, X., Luo, X., Wang, P., and Qiao, Z. Out-of-distribution graph models merging. ArXiv, abs/2506.03674, 2025b.
- Weimar, M., Rachbauer, L. M., Starshynov, I., Faccio, D., Adilova, L., Bouchet, D., and Rotter, S. Fisher information flow in artificial neural networks. ArXiv, abs/2509.02407, 2025.
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Lopes, R. G., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., and Schmidt, L. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In International Conference on Machine Learning (ICML), pp. 23965–23998, 2022.
- Xiong, H. and Yang, C. Test-time adaptation with robust dual-stream perturbation for stable agent deployment in dynamic scenarios. 2024 27th International Conference on Computer Supported Cooperative Work in Design (CSCWD), pp. 477–482, 2024.
- Xiong, H., Yang, Q., Wang, C., Zhong, T., Zhao, Z., and Pu, Y. Layer-wise parameter robustness for continual test-time adaptation. 2025 IEEE International Conference on Multimedia and Expo (ICME), pp. 1–6, 2025.
- Yadav, P., Dare, D., Kilgour, D., Wortsman, M., Ravikumar, P., and Hajishirzi, H. Ties-merging: Resolving interference when merging models. In Neural Information Processing Systems (NeurIPS), 2023a.
- Yadav, P., Tam, D., Choshen, L., Raffel, C., and Bansal, M. Ties-merging: Resolving interference when merging models. Advances in Neural Information Processing Systems 36, 2023b.
- Yang, D., Choi, S., and Lee, H. Mitigating the feedback trap in test-time model merging. In International Conference on Machine Learning (ICML), 2024.

- Yang, X., Li, F., Li, T., Ji, W., and Liang, Y. Elastic weight consolidation continual learning based signal detection in multiple channel mimo system. 2021 IEEE/CIC International Conference on Communications in China (ICCC), pp. 534–539, 2021.
- Ye, P., Huang, C., Shen, M., Chen, T., Huang, Y., and Ouyang, W. Dynamic model merging with mixture of weights. IEEE Transactions on Circuits and Systems for Video Technology, 35:7925–7939, 2025.
- You, F., Li, J., and Zhao, Z. Test-time batch statistics calibration for covariate shift. ArXiv, abs/2110.04065, 2021.
- Yuan, L., Xie, B., and Li, S. Robust test-time adaptation in dynamic scenarios. 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 15922–15932, 2023.
- Zhang, C., Yao, X., Shi, C., and Gu, M. Kronecker-factored approximate curvature with adaptive learning rate for optimizing model-agnostic meta-learning. Multimedia Systems, 29:3169 – 3177, 2023.
- Zhou, Z., Chen, Z., Chen, Y., Zhang, B., and Yan, J. Cross-task linearity emerges in the pretraining-finetuning paradigm. ArXiv, abs/2402.03660, 2024a.
- Zhou, Z., Chen, Z., Chen, Y., Zhang, B., and Yan, J. On the emergence of cross-task linearity in pretraining-finetuning paradigm. pp. 61854–61884, 2024b.
- Zhou, Z., Pi, L., Xu, H., Zhou, S., and Kong, Y. Test-time training with invariant graph learning for out-of-distribution generalization. Engineering Research Express, 7, 2025.
- Zhu, Y., Xia, R., and Zhang, J. Dppa: Pruning method for large language model to model merging. ArXiv, abs/2403.02799, 2024.

A. Mathematical Derivation of Soft BN Buffer Fusion

In this section, we provide the full mathematical derivation for our proposed Soft BN Buffer Fusion (Moment Matching under a Mixture-of-Gaussians model) presented in Section 3.1, Equations 3 and 4.

Let the feature activation z of a Batch Normalization layer be modeled as a mixture distribution over the K available task-specific expert networks. Let w_k denote the soft posterior probability of expert k being the active model for the current batch (computed via Eq. 2). Under this mixture model, we assume that the conditional activation of z given expert k follows a Gaussian distribution:

$$z \mid \text{expert } k \sim \mathcal{N}(\mu_k, \sigma_k^2), \quad (13)$$

where μ_k and σ_k^2 are the running mean and variance stored in the Batch Normalization buffer of expert k .

The probability density function (PDF) of the mixed activation z is given by the Mixture of Gaussians (MoG):

$$p(z) = \sum_{k=1}^K w_k \mathcal{N}(z; \mu_k, \sigma_k^2), \quad (14)$$

where $\sum_{k=1}^K w_k = 1$.

A.1. Derivation of the Fused Mean μ_{fused}

The expectation (mean) of the mixed activation z under the MoG distribution $p(z)$ is computed as:

$$\mu_{\text{fused}} = \mathbb{E}[z] = \int_{-\infty}^{\infty} z p(z) dz = \int_{-\infty}^{\infty} z \left(\sum_{k=1}^K w_k \mathcal{N}(z; \mu_k, \sigma_k^2) \right) dz. \quad (15)$$

By the linearity of integration and summation:

$$\mu_{\text{fused}} = \sum_{k=1}^K w_k \left(\int_{-\infty}^{\infty} z \mathcal{N}(z; \mu_k, \sigma_k^2) dz \right). \quad (16)$$

Since the integral of z over the Gaussian density $\mathcal{N}(z; \mu_k, \sigma_k^2)$ is its mean μ_k :

$$\mu_{\text{fused}} = \sum_{k=1}^K w_k \mu_k. \quad (17)$$

This proves Equation 3.

A.2. Derivation of the Fused Variance σ_{fused}^2

The variance of the mixed activation z is defined as:

$$\sigma_{\text{fused}}^2 = \text{Var}(z) = \mathbb{E}[z^2] - (\mathbb{E}[z])^2 = \mathbb{E}[z^2] - \mu_{\text{fused}}^2. \quad (18)$$

First, we compute the second moment $\mathbb{E}[z^2]$:

$$\mathbb{E}[z^2] = \int_{-\infty}^{\infty} z^2 p(z) dz = \sum_{k=1}^K w_k \left(\int_{-\infty}^{\infty} z^2 \mathcal{N}(z; \mu_k, \sigma_k^2) dz \right). \quad (19)$$

The second moment of a Gaussian random variable is given by $\sigma_k^2 + \mu_k^2$. Substituting this:

$$\mathbb{E}[z^2] = \sum_{k=1}^K w_k (\sigma_k^2 + \mu_k^2). \quad (20)$$

Substituting this back into the expression for $\text{Var}(z)$:

$$\sigma_{\text{fused}}^2 = \sum_{k=1}^K w_k (\sigma_k^2 + \mu_k^2) - \mu_{\text{fused}}^2. \quad (21)$$

Now, let us expand our proposed fused variance formulation (Equation 4):

$$\text{Proposed Var} = \sum_{k=1}^K w_k (\sigma_k^2 + (\mu_k - \mu_{\text{fused}})^2). \quad (22)$$

Expanding the quadratic term:

$$\text{Proposed Var} = \sum_{k=1}^K w_k (\sigma_k^2 + \mu_k^2 - 2\mu_k \mu_{\text{fused}} + \mu_{\text{fused}}^2). \quad (23)$$

Distributing the summation and the mixture weights w_k :

$$\text{Proposed Var} = \sum_{k=1}^K w_k (\sigma_k^2 + \mu_k^2) - 2\mu_{\text{fused}} \left(\sum_{k=1}^K w_k \mu_k \right) + \mu_{\text{fused}}^2 \left(\sum_{k=1}^K w_k \right). \quad (24)$$

Recall that $\sum_{k=1}^K w_k \mu_k = \mu_{\text{fused}}$ and $\sum_{k=1}^K w_k = 1$. Substituting these properties:

$$\text{Proposed Var} = \sum_{k=1}^K w_k (\sigma_k^2 + \mu_k^2) - 2\mu_{\text{fused}}^2 + \mu_{\text{fused}}^2. \quad (25)$$

Simplifying the terms:

$$\text{Proposed Var} = \sum_{k=1}^K w_k (\sigma_k^2 + \mu_k^2) - \mu_{\text{fused}}^2. \quad (26)$$

This is mathematically identical to Equation 21, proving that our Soft BN Buffer Fusion computes the exact statistical moments of the mixture distribution. This completes the proof of Equation 4.

B. Experimental and Implementation Details

In this section, we provide comprehensive specifications of our experimental setup, neural network architectures, and hyper-parameters to facilitate full reproducibility.

B.1. Architectural Details

We employ a modified ResNet-18 model architecture (He et al., 2016) as our backbone model for both expert neural networks. To accommodate 1-channel grayscale image inputs (from the MNIST, KMNIST, and Fashion-MNIST datasets) while preserving pre-trained weights from ImageNet, we apply a channel reduction surgery:

- **Input Conv Layer Surgery:** The standard ImageNet ResNet-18 first convolutional layer (conv1) expects 3 input channels with weight dimensions of $[64, 3, 7, 7]$. We construct a new single-channel convolutional layer with dimensions $[64, 1, 7, 7]$. The weights are initialized by summing the pretrained weights across the channel dimension: $W_{\text{new}} = \sum_{c=1}^3 W_{\text{old},c}$. This allows the network to process grayscale inputs while retaining the learned spatial features.
- **Classification Head:** The final fully connected layer (fc) is replaced with a linear layer mapping from 512 features to 10 classes: `nn.Linear(512, 10)`.

B.2. Expert Pre-training Hyper-parameters

Each expert model is trained independently on the training split of its respective dataset (MNIST and KMNIST). The training details are:

- Preprocessing: Grayscale input images are converted to PyTorch tensors and normalized to the range $[-1.0, 1.0]$ using a mean of 0.5 and a standard deviation of 0.5.
- Optimization: We train using the AdamW optimizer with a learning rate of 1×10^{-3} and weight decay of 1×10^{-4} .
- Duration and Batch Size: Each expert is trained for 3 epochs with a batch size of 256.
- Validation Performance: The MNIST expert achieves a validation accuracy of 98.81%, and the KMNIST expert achieves 95.34%.

B.3. FDF-DPA Deployment Hyper-parameters

For deployment on the non-stationary stream, FDF-DPA operates under the following default parameters:

- Test Batch Size: 64 samples per batch.
- Entropy Threshold τ_N : 0.70 (calibrated to the natural logarithm scale).
- Update Momentum α_{ema} : 0.1.
- Prototype Blending Coefficient α_{proto} : 0.95.
- Preconditioning Base Learning Rate η : 0.005.
- Damping Coefficient β : 0.5.
- Bayes Temperature scaling γ : 5.0.
- Anchored Feature Extraction Layers: conv1, bn1, layer1, and layer2.